

# CCC Recommendation Engine

Technical Blueprint for Vibe Coding

Working Capital Intelligence Platform · SME & Mid-Market Manufacturers

End-to-end process flow · Architecture decisions · Technical difficulties & solutions · Agent build plan

|        |             |               |           |
|--------|-------------|---------------|-----------|
| 6      | 4           | 3             | 1         |
| Agents | Data layers | Core formulas | Dashboard |

## 1. Product Overview & Goal

The CCC Recommendation Engine is the core intelligence layer of a Working Capital Intelligence Platform designed for SME and mid-market manufacturers. Its sole objective: compute the Cash Conversion Cycle from live transactional data, benchmark it against industry norms, and emit a ranked set of actionable recommendations that reduce the cycle —

freeing up locked working capital without requiring a treasury team.

The Cash Conversion Cycle formula

$$CCC = DIO + DSO - DPO$$

DIO (Days Inventory Outstanding) + DSO (Days Sales Outstanding) – DPO (Days Payable Outstanding)

Target customer profile

| Attribute         | Detail                                                                |
|-------------------|-----------------------------------------------------------------------|
| Industry segment  | SME and mid-market manufacturers, distributors, traders               |
| Company size      | 10–500 employees, Rs 5Cr–500Cr annual revenue                         |
| Primary users     | CFO, Finance Controller, Accounts Manager                             |
| Pain point        | No visibility into why cash feels tight despite strong sales          |
| Current tools     | Tally ERP, Zoho Books, Excel — none actively recommend action         |
| Value proposition | First tool that tells you exactly which lever to pull and by how much |

## 2. End-to-End Process Flow

The pipeline has six discrete stages. Each stage has a clean input contract and output contract so individual agents can build them independently without coordination overhead.

| Stage 1 | Data Ingestion                                                                           |
|---------|------------------------------------------------------------------------------------------|
| Input   | User uploads CSV / connects Tally / manually enters invoices, inventory, sales.          |
| Output  | Normalised rows in the universal schema (Invoice, InventoryItem, SalesRecord).           |
| Stage 2 | Data Normalisation Layer                                                                 |
| Input   | Raw records from any source are mapped to typed internal objects with validation.        |
| Output  | Validated domain objects ready for calculation — rejects malformed rows with error list. |
| Stage 3 | DIO / DSO / DPO Calculator                                                               |
| Input   | Aggregates normalised data over a user-selected period (30 / 60 / 90 days).              |
| Output  | Three numeric values: DIO, DSO, DPO, each with a trend delta vs. prior period.           |
| Stage 4 | CCC Scoring & Benchmarking                                                               |
| Input   | Combines DIO + DSO - DPO. Fetches industry benchmark. Scores relative performance.       |
| Output  | CCC score, benchmark, gap, percentile band (Top 25% / Median / Bottom 25%).              |
| Stage 5 | Recommendation Engine                                                                    |
| Input   | Rule-based engine maps each gap dimension to a library of strategic actions.             |
| Output  | Priority-ranked list of recommendations with estimated days-reduction impact.            |
| Stage 6 | CFO Dashboard                                                                            |
| Input   | React frontend renders metric cards, a 6-period trend chart, and the action panel.       |
| Output  | Interactive dashboard; user can mark actions as In Progress or Dismissed.                |

## 3. Architectural Decisions by Layer

### Frontend — UI Layer

Stack: Next.js 14 (App Router) + React 18 + Tailwind CSS + shadcn/ui

#### Why Next.js over plain React?

Server Components reduce bundle size for data-heavy dashboards. RSC fetches CCC data server-side so the page loads with data pre-rendered — critical for CFO dashboards where first paint = first trust.

#### Component pattern

Compound component pattern for the MetricCard family (MetricCard, MetricCard.Trend, MetricCard.Badge). Keeps the dashboard page clean and composable.

#### State management

Zustand for local UI state (active period selector, dismissed recommendations). No Redux — overkill for a focused dashboard. Server state lives in React Query (TanStack Query).

#### Chart library

Recharts — ships lighter than Chart.js for the trend line use case. ResponsiveContainer handles the CFO dashboard's varying column widths without manual resize logic.

### API Layer

Stack: Next.js Route Handlers (App Router) + Zod validation + typed responses

#### Pattern: thin controllers, fat services

Route handlers validate input with Zod and call a service function. Zero business logic lives in the handler. This lets Agent 4 build routes without knowing how the CCC engine works internally.

#### Validation-first design

Every POST endpoint parses the request body with a Zod schema before any DB call. Invalid payloads return a structured 400 with field-level errors — important for the CSV import flow.

#### Response envelope

All responses follow { data, error, meta } shape. meta includes calculatedAt, periodDays, and recordCount. Clients never need to guess what came back.

#### Rate limiting

Upstash Redis-based rate limiter on the /api/ccc/calculate endpoint. CCC computation is expensive (full table scan) — limit to 10 calls per company per hour.

### Business Logic Layer

Stack: /lib/ccc-engine — pure TypeScript functions, zero side effects

#### Pure function architecture

calculateDIO(), calculateDSO(), calculateDPO(), calculateCCC() are all pure functions: same inputs always produce the same output. No DB calls, no network calls, no side effects. This makes unit testing trivial and lets the recommendation

engine call them in isolation.

### Separation from ORM

The engine receives plain typed objects, not Prisma models. A mapper function converts Prisma models to engine input types at the service boundary. This decouples the engine from the DB schema.

### Recommendation rule engine

Table-driven rules. Each rule is { id, condition: (cccResult) => boolean, priority: fn, recommendation: fn }. Adding a new recommendation means adding one row to the rules table — no if-else chains, no switch statements.

### Benchmarking data

Industry benchmarks stored as a JSON config file keyed by MSIC industry code. Loaded at startup, not fetched per request. Median CCC values sourced from published RBI and SIDBI SME reports.

## Data Layer

**Stack: PostgreSQL + Prisma ORM + connection pooling via PgBouncer**

### Why PostgreSQL over MongoDB?

CCC computation requires aggregate queries over date ranges (AVG, SUM, GROUP BY). Relational data with clear foreign keys (Invoice -> Customer, Payment -> Vendor) is a better fit than document storage. SQL window functions simplify the DIO trend calculation.

### Prisma as the ORM

Prisma generates TypeScript types from the schema — Agent 3's engine types and Agent 2's schema stay in sync automatically. Prisma Migrate handles all schema changes.

### CCCSnapshot table

Every /api/ccc/calculate call writes a snapshot row (companyId, periodStart, periodEnd, dio, dso, dpo, ccc, benchmarkCCC). This gives the trend chart data without re-running the calculation.

### Index strategy

Composite index on (companyId, invoiceDate) for Invoice table. Composite index on (companyId, periodStart) for CCCSnapshot. Both are the hot paths in every calculation query.

## Infrastructure Layer

**Stack: Vercel (frontend + API) + Supabase (Postgres) + Upstash (Redis)**

### Deployment target

Vercel for Next.js — zero config, automatic preview deployments per PR. Edge functions for rate limiting middleware. Supabase for managed Postgres with built-in connection pooling.

### Environment separation

Three environments: dev (local Postgres via Docker), staging (Supabase staging project), production (Supabase production). DATABASE\_URL switches per environment via .env files.

### File uploads

CSV uploads go to Vercel Blob (or S3-compatible storage). A background job (Vercel Cron) processes the file and populates the DB. The user sees a 'Processing' badge until done.

## 4. Technical Difficulties & Solutions

These are the hard problems you will hit during the build — most vibe-coding sessions stall on exactly these. Each entry includes the root cause, the symptom you will observe, and the precise solution to implement.

| D1       | Partial period data skews CCC calculation                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem  | If a company starts using the app mid-month, the first period has fewer days of data than the selected window (e.g. 15 days of invoices for a 30-day DIO calc). DIO comes out abnormally low.                                                                                                                                                                                                                                                            |
| Solution | Enforce a minimum data completeness threshold. If <code>recordedDays &lt; 0.8 * periodDays</code> , mark the snapshot as PARTIAL and show a warning badge on the dashboard. Do not suppress the result — show it with context. Add a <code>dataCompleteness: number</code> field to <code>CCCSnapshot</code> .                                                                                                                                           |
| D2       | Invoices without payment dates break DPO                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Problem  | Vendors sometimes send invoices that are not yet paid. These have no <code>paymentDate</code> . If your DPO query naively averages ( <code>paymentDate - invoiceDate</code> ), NULL rows are silently excluded — DPO is understated and the CCC looks better than it is.                                                                                                                                                                                 |
| Solution | Use two DPO figures: <code>paidDPO</code> (completed payments only) and <code>projectedDPO</code> (includes outstanding invoices, using today as the pseudo-payment date). Show both on the dashboard. Alert the user when <code>projectedDPO &gt; paidDPO</code> by more than 10 days — it signals overdue payables building up.                                                                                                                        |
| D3       | COGS data is missing or inconsistent                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Problem  | $DIO = (\text{Average Inventory} / \text{COGS}) * \text{Days}$ . Most SMEs track inventory value but not COGS separately. They often conflate purchase cost and COGS, or have no COGS figure at all.                                                                                                                                                                                                                                                     |
| Solution | Derive COGS from sales records if not provided directly: $COGS = \text{Revenue} - \text{Gross Profit}$ . If Gross Profit is also unavailable, accept a user-entered Gross Margin % as a fallback and compute $COGS = \text{Revenue} * (1 - \text{grossMarginPct})$ . Flag derived COGS in the result object with <code>cogsSource: 'direct'   'derived'   'estimated'</code> .                                                                           |
| D4       | Recommendation engine produces irrelevant advice                                                                                                                                                                                                                                                                                                                                                                                                         |
| Problem  | A company with DPO already at 60 days gets a recommendation to 'extend payment terms' — but they are already at the industry max and pushing further damages vendor relationships.                                                                                                                                                                                                                                                                       |
| Solution | Cap each recommendation by a feasibility ceiling. Store industry-specific DPO ceilings in the benchmark config (e.g. manufacturing: <code>maxDPO 65 days</code> ). The rule condition checks: <code>currentDPO &lt; benchmark.maxDPO - 5</code> before emitting extend-DPO recommendations. Also add a <code>companyContext</code> object where users can flag 'Key vendor — do not delay payment' to suppress DPO recommendations for specific vendors. |

| D5       | CSV import fails silently on malformed rows                                                                                                                                                                                                                                                                                              |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem  | A user uploads a 500-row CSV. 47 rows have missing dueDate or invalid amount format. The import job skips them silently. The user sees 453 records imported with no idea 47 were dropped.                                                                                                                                                |
| Solution | Implement row-level validation with Zod during import. Collect all errors into an ImportResult object: { imported: number, skipped: number, errors: { row: number, field: string, message: string }[] }. Show a detailed error report UI after every import. Never silently drop rows.                                                   |
| D6       | Trend chart shows false improvement after data correction                                                                                                                                                                                                                                                                                |
| Problem  | A user corrects a backdated invoice. This changes historical DIO/DSO for a past period. The CCCSnapshot for that period now has stale values and the trend chart shows a misleading dip.                                                                                                                                                 |
| Solution | Add a recalculate flag to CCCSnapshot. When historical data changes, mark affected snapshots as STALE. On dashboard load, check for STALE snapshots and either re-compute them in the background or show a 'Data updated — refreshing analysis' notice. Never silently show stale trend data.                                            |
| D7       | Type drift between Prisma models and engine types                                                                                                                                                                                                                                                                                        |
| Problem  | Agent 2 updates the schema (e.g. renames invoiceDate to issuedAt). Agent 3's engine types still reference invoiceDate. TypeScript catches it at compile time but only if the mapper is also updated. In a vibe-coding session, this often gets missed.                                                                                   |
| Solution | Define all domain types in /lib/ccc-engine/types.ts as the single source of truth. The Prisma schema uses Prisma's own naming; the mapper in /lib/mappers/invoice.mapper.ts is the only file that bridges them. Any schema rename breaks exactly one file (the mapper) — no hunt across the codebase.                                    |
| D8       | N+1 query problem in DIO calculation                                                                                                                                                                                                                                                                                                     |
| Problem  | Naive implementation loads all InventoryItem records, then for each item loads its SalesRecord to get COGS. With 500 SKUs this fires 501 DB queries per CCC calculation — catastrophic latency.                                                                                                                                          |
| Solution | Write a single aggregate SQL query using Prisma's groupBy and _sum aggregations. Pre-join inventory and sales in one query, aggregate at the DB level. Use Prisma's queryRaw as a last resort for complex window function queries. Target: CCC calculation completes in under 200ms for companies with up to 10,000 transaction records. |

## 5. Database Schema — Prisma Models

```
model Company { id String @id @default(cuid()) name String industry String createdAt DateTime @default(now()) invoices Invoice[] inventoryItems InventoryItem[] salesRecords SalesRecord[] cccSnapshots CCCSnapshot[] } model Invoice { id String @id @default(cuid()) companyId String company Company @relation(...) type InvoiceType // AP | AR counterpartyName String amount Decimal currency String @default('INR') invoiceDate DateTime dueDate DateTime paymentDate DateTime? status PaymentStatus // PAID | PENDING | OVERDUE @@index([companyId, invoiceDate]) } model InventoryItem { id String @id @default(cuid()) companyId String sku String description String avgValueOnHand Decimal periodStart DateTime periodEnd DateTime cogsForPeriod Decimal? cogsSource CogsSource @default(DIRECT) @@index([companyId, periodStart]) } model SalesRecord { id String @id @default(cuid()) companyId String periodStart DateTime periodEnd DateTime revenue Decimal grossMarginPct Decimal? cogs Decimal? } model CCCSnapshot { id String @id @default(cuid()) companyId String periodStart DateTime periodEnd DateTime periodDays Int dio Decimal dso Decimal dpo Decimal ccc Decimal benchmarkCCC Decimal? dataCompleteness Decimal status SnapshotStatus // COMPLETE | PARTIAL | STALE calculatedAt DateTime @default(now()) @@index([companyId, periodStart]) }
```

### Key design decisions

| Decision                     | Rationale                                                                                                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Invoice.type = AP   AR       | Single invoice table handles both payables and receivables. Simplifies querying across both sides and avoids join complexity. Filter by type at the query level.             |
| paymentDate is nullable      | Null = not yet paid. This drives the projectedDPO vs paidDPO split (Difficulty D2).                                                                                          |
| cogsSource enum              | Tracks data quality. DIRECT = user-entered. DERIVED = computed from margin. ESTIMATED = user provided a gross margin %. Powers the data quality badge on the dashboard.      |
| CCCSnapshot stores benchmark | Snapshot the benchmark at calculation time — not recalculated later. Benchmark config may update; historical comparisons should reflect what the benchmark was at that time. |
| SnapshotStatus = STALE       | Enables the historical data correction workflow (Difficulty D6) without deleting snapshots.                                                                                  |



## 6. Core Engine Types — `//lib/ccc-engine/types.ts`

```
// Input types (engine receives these – decoupled from Prisma)
interface InvoiceRecord { id: string;
  type: 'AP' | 'AR'; amount: number; invoiceDate: Date; dueDate: Date; paymentDate: Date | null; }
interface InventoryRecord { sku: string; avgValueOnHand: number; cogsForPeriod: number | null;
  cogsSource: 'direct' | 'derived' | 'estimated'; }
interface SalesRecord { periodStart: Date; periodEnd: Date; revenue: number; cogs: number | null; grossMarginPct: number | null; }
interface CCCInput { periodDays: number; invoices: InvoiceRecord[]; inventory: InventoryRecord[]; sales: SalesRecord[];
  industryCode: string; }
// Output types
interface ComponentResult { value: number; trendDelta: number; dataCompleteness: number; }
interface CCCResult { dio: ComponentResult; dso: ComponentResult; dpo: ComponentResult; ccc: number; benchmark: number | null; gapToBenchmark: number | null; percentileBand:
  'top25' | 'median' | 'bottom25' | null; recommendations: Recommendation[]; }
interface Recommendation { id: string; dimension: 'DIO' | 'DSO' | 'DPO'; priority: 'HIGH' | 'MEDIUM' | 'LOW'; title: string;
  description: string; estimatedDaysReduction: number; actionSteps: string[]; }
```

## 7. Recommendation Rule Library

Rules are evaluated after every CCC calculation. Each rule has a condition that fires based on the CCC result, a priority score function, and a template for the recommendation output. Rules are additive — multiple rules can fire for the same calculation.

| Rule                                             | Trigger                                                                                                              | Recommended action                                                                                                                          |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>R1 — Invoice aging acceleration</b>           | Dimension: <b>DSO</b> · Priority: <b>HIGH</b><br>Condition: <i>DSO &gt; benchmark.dso + 10</i>                       | Implement automated payment reminders at 7, 14, and 21 DPD. Offer 1.5% early payment discount for settlement within 15 days.                |
| <b>R2 — Prioritise overdue collections</b>       | Dimension: <b>DSO</b> · Priority: <b>HIGH</b><br>Condition: <i>Any AR invoice &gt; 60 days overdue</i>               | Identify top 5 overdue invoices by value. Escalate to direct CFO-to-CFO contact. Consider invoice discounting for invoices over 90 days.    |
| <b>R3 — Reduce slow-moving inventory</b>         | Dimension: <b>DIO</b> · Priority: <b>HIGH</b><br>Condition: <i>DIO &gt; benchmark.dio + 15</i>                       | Flag SKUs with DIO > 90 days. Initiate clearance pricing. Switch to just-in-time ordering for top 3 slow-moving categories.                 |
| <b>R4 — Tighten reorder point calculation</b>    | Dimension: <b>DIO</b> · Priority: <b>MEDIUM</b><br>Condition: <i>DIO &gt; benchmark.dio + 5</i>                      | Review reorder points for top 20 SKUs by value. Reduce safety stock by 15% for SKUs with reliable supplier lead times.                      |
| <b>R5 — Negotiate extended payment terms</b>     | Dimension: <b>DPO</b> · Priority: <b>MEDIUM</b><br>Condition: <i>DPO &lt; benchmark.dpo - 10 AND DPO &lt; maxDPO</i> | Approach top 3 vendors (by spend) to negotiate Net 45 from current Net 30. Offer consistent on-time payment record as leverage.             |
| <b>R6 — Evaluate early payment discount ROI</b>  | Dimension: <b>DPO</b> · Priority: <b>LOW</b><br>Condition: <i>Early payment discounts available</i>                  | Calculate annualised return on 2/10 Net 30 terms (36.7% APR). Accept discount only if cash yield exceeds current borrowing cost.            |
| <b>R7 — Introduce advance payment policy</b>     | Dimension: <b>DSO</b> · Priority: <b>MEDIUM</b><br>Condition: <i>DSO &gt; 45 AND no upfront deposits recorded</i>    | Require 25-30% advance for orders above Rs 5L from customers with DSO > 45 days. Structure as a standard onboarding term for new customers. |
| <b>R8 — Review procurement forecast accuracy</b> | Dimension: <b>DIO</b> · Priority: <b>LOW</b><br>Condition: <i>DIO trending up 3+ consecutive periods</i>             | Compare purchase orders against actual consumption for last quarter. Reduce forward purchase commitments by 20% and reassess monthly.       |

## 8. Agent Build Plan — Vibe Coding Sequence

Feed each agent card as a prompt to your IDE agent (Cursor, Windsurf, Claude Code). Each agent's output becomes context for the next. Do not skip the sequence — later agents depend on types and file paths established by earlier ones.

| Agent 1     | Project scaffold                                                                                                                                                                                                                                                                                                                                                               |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Bootstrap the monorepo with all configuration. No business logic — only skeleton.                                                                                                                                                                                                                                                                                              |
| Creates     | Next.js 14 App Router project with TypeScript strict mode. Tailwind CSS + shadcn/ui. Prisma CLI installed. <code>/lib/ccs-engine</code> , <code>/lib/mappers</code> , <code>/components</code> , <code>/types</code> directories. <code>.env.example</code> with <code>DATABASE_URL</code> , <code>DIRECT_URL</code> . <code>eslint</code> + <code>prettier</code> configured. |
| Prompt seed | <i>"Create a Next.js 14 App Router project with TypeScript strict, Tailwind CSS, shadcn/ui, and Prisma. Create these empty directories: <code>/lib/ccs-engine</code>, <code>/lib/mappers</code>, <code>/components/dashboard</code>. Add <code>.env.example</code>. Do not write any business logic yet."</i>                                                                  |

| Agent 2     | Prisma schema + seed                                                                                                                                                                                                                                                                                                                                   |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Define the full database schema and seed it with realistic manufacturer sample data.                                                                                                                                                                                                                                                                   |
| Creates     | <code>schema.prisma</code> with <code>Company</code> , <code>Invoice (AP+AR)</code> , <code>InventoryItem</code> , <code>SalesRecord</code> , <code>CCCSnapshot</code> models. All indexes defined. Seed script populates 90 days of data for a fictional auto parts manufacturer.                                                                     |
| Prompt seed | <i>"Using the schema from Section 5 of the blueprint, implement <code>schema.prisma</code> with all models and indexes. Then write <code>prisma/seed.ts</code> that creates a sample company with 90 days of AP invoices, AR invoices, inventory records, and sales records. Run <code>prisma migrate dev</code> and <code>prisma db seed</code>."</i> |

| Agent 3     | CCC calculator engine                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Build the pure TypeScript calculation functions. No framework dependencies.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Creates     | <code>/lib/ccs-engine/types.ts</code> (all types from Section 6). <code>/lib/ccs-engine/calculator.ts</code> with <code>calculateDIO</code> , <code>calculateDSO</code> , <code>calculateDPO</code> , <code>calculateCCC</code> . <code>/lib/ccs-engine/benchmarks.json</code> . Unit tests for all four functions.                                                                                                                                                                                                                                                                                |
| Prompt seed | <i>"Implement the types from Section 6 in <code>/lib/ccs-engine/types.ts</code>. Then implement <code>/lib/ccs-engine/calculator.ts</code> with pure functions: <code>calculateDIO(inventory, sales, periodDays)</code>, <code>calculateDSO(arInvoices, revenue, periodDays)</code>, <code>calculateDPO(apInvoices, cogs, periodDays)</code>, <code>calculateCCC(dio, dso, dpo)</code>. Include <code>trendDelta</code> and <code>dataCompleteness</code> in each <code>ComponentResult</code>. Write tests for edge cases: empty arrays, null <code>paymentDate</code>, partial period data."</i> |

| Agent 4 | Recommendation engine |
|---------|-----------------------|
|---------|-----------------------|

|             |                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Build the rule table and the function that maps a CCCResult to a Recommendation array.                                                                                                                                                                                                                                                                                                |
| Creates     | /lib/ccc-engine/recommendations.ts with all 8 rules from Section 7. Each rule is a typed object. generateRecommendations(result: CCCResult): Recommendation[] is the public API.                                                                                                                                                                                                      |
| Prompt seed | <i>"Using the types from types.ts and the 8 rules from Section 7, implement /lib/ccc-engine/recommendations.ts. Define a RecommendationRule interface with id, condition, priority, and recommend fields. Implement all 8 rules as a typed array. Export generateRecommendations() that filters rules by condition, sorts by priority, and returns typed Recommendation objects."</i> |

| Agent 5     | API routes                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Implement all Next.js Route Handlers with Zod validation and typed responses.                                                                                                                                                                                                                                                                                                                                                                                  |
| Creates     | POST /api/invoices, GET /api/invoices. POST /api/inventory. POST /api/sales. GET /api/ccc/calculate — runs engine, saves snapshot, returns CCCResult + recommendations. GET /api/ccc/history — returns last 6 CCCSnapshots for trend chart.                                                                                                                                                                                                                    |
| Prompt seed | <i>"Using schema.prisma from Agent 2 and types.ts from Agent 3, implement the API routes. Every route validates input with Zod before touching the DB. GET /api/ccc/calculate must: query DB for the requested period, call the mapper functions to convert Prisma models to engine input types, call calculateCCC(), call generateRecommendations(), save a CCCSnapshot, and return the full CCCResult. Use the { data, error, meta } response envelope."</i> |

| Agent 6     | CFO dashboard                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role        | Build the React dashboard that renders all CCC data and recommendations.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Creates     | /app/dashboard/page.tsx as a Server Component that fetches initial data. MetricCard component (DIO, DSO, DPO, CCC with trend delta and benchmark gap). Recharts LineChart for 6-period trend. RecommendationsPanel with priority sort and dismiss/mark-in-progress actions.                                                                                                                                                                                                                                                                                    |
| Prompt seed | <i>"Using the API route shapes from Agent 5 and CCCResult type from Agent 3, build the CFO dashboard. Create /components/dashboard/MetricCard.tsx — accepts label, value, unit, trendDelta, benchmarkGap props. Create /components/dashboard/RecommendationsPanel.tsx — renders recommendations sorted by priority with HIGH cards in red-tinted border, MEDIUM amber, LOW gray. Create /components/dashboard/CCCTrendChart.tsx using Recharts ResponsiveContainer and LineChart with three lines (DIO, DSO, DPO) and a reference line for benchmark CCC."</i> |

## 9. API Contract Reference

| Meth<br>od | Endpoint           | Request body / params                                                             | Response shape                                                       |
|------------|--------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| POST       | /api/invoices      | { type: 'AP' 'AR', counterpartyName, amount, invoiceDate, dueDate, paymentDate? } | { data: Invoice, meta: { recordId } }                                |
| GET        | /api/invoices      | ?type=AP AR&from;=ISO_DATE&to;=ISO_DATE                                           | { data: Invoice[], meta: { count, periodDays } }                     |
| POST       | /api/inventory     | { sku, description, avgValueOnHand, periodStart, periodEnd, cogsForPeriod? }      | { data: InventoryItem }                                              |
| POST       | /api/sales         | { periodStart, periodEnd, revenue, grossMarginPct? }                              | { data: SalesRecord }                                                |
| GET        | /api/ccc/calculate | ?periodDays=30 60 90&companyId;=string                                            | { data: CCCResult, meta: { calculatedAt, periodDays, recordCount } } |
| GET        | /api/ccc/history   | ?companyId=string&limit;=6                                                        | { data: CCCSnapshot[], meta: { count } }                             |
| POST       | /api/import/csv    | multipart/form-data: file (CSV), type ('invoices' 'inventory' 'sales')            | { data: ImportResult: { imported, skipped, errors[] } }              |

## 10. Final File Structure

```
/ ■■■ app/ ■ ■■■ api/ ■ ■ ■■■ invoices/route.ts ← Agent 5 ■ ■ ■■■ inventory/route.ts ← Agent 5 ■ ■ ■■■ sales/route.ts ← Agent 5 ■ ■ ■■■ ccc/ ■ ■ ■■■ calculate/route.ts ← Agent 5 (core endpoint) ■ ■ ■■■ history/route.ts ← Agent 5 ■ ■ ■■■ import/csv/route.ts ← Agent 5 ■ ■■■ dashboard/ ■ ■■■ page.tsx ← Agent 6 (Server Component) ■■■ components/ ■ ■■■ dashboard/ ■ ■■■ MetricCard.tsx ← Agent 6 ■ ■■■ CCTrendChart.tsx ← Agent 6 ■ ■■■ RecommendationsPanel.tsx ← Agent 6 ■■■ lib/ ■ ■■■ ccc-engine/ ■ ■ ■■■ types.ts ← Agent 3 (source of truth) ■ ■ ■■■ calculator.ts ← Agent 3 ■ ■ ■■■ recommendations.ts ← Agent 4 ■ ■ ■■■ benchmarks.json ← Agent 3 ■ ■■■ mappers/ ■ ■ ■■■ invoice.mapper.ts ← Agent 5 ■ ■ ■■■ inventory.mapper.ts ← Agent 5 ■ ■ ■■■ sales.mapper.ts ← Agent 5 ■ ■■■ db.ts ← Agent 1 (Prisma client) ■■■ prisma/ ■ ■■■ schema.prisma ← Agent 2 ■ ■■■ seed.ts ← Agent 2 ■ ■■■ types/ ■ ■■■ api.ts ← Agent 5 (response envelopes) ■■■ .env.example ← Agent 1
```